

Worksheet No. - 9

Student Name: Ankush kumar

Branch: MCA

Semester: 2nd

Subject Name: TECHNICAL TRAINING

UID: 25MCA20305

Section/Group: 1A

Date of Performance: 1th Apr, 2026

Subject Code: 25CAP-652

Aim/Overview of the practical:

Apply trigger concepts to enforce constraints and automate tasks

Objective:

To apply the concept of Triggers in database operations in order to perform tasks like --
insertion, updating, deletion, and retrieval
of data efficiently, securely, within the database system.

Input/Apparatus Used:

- PostgreSQL
- pgAdmin

Procedure/Algorithm/Code :

```
-- =====  
  
-- DROP TABLES (cleanup)  
  
-- =====  
  
DROP TABLE IF EXISTS users CASCADE;  
DROP TABLE IF EXISTS user_Logs;  
DROP TABLE IF EXISTS update_Logs;  
DROP TABLE IF EXISTS delete_Logs;  
  
  
-- =====  
  
-- MAIN TABLE  
  
-- =====
```

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL,  
    created_at TIMESTAMP,  
    updated_at TIMESTAMP  
);  
  
-- =====  
-- LOG TABLES  
-- =====  
  
CREATE TABLE user_Logs (  
    Log_id SERIAL PRIMARY KEY,  
    user_id INT,  
    action TEXT,  
    Log_time TIMESTAMP  
);  
  
CREATE TABLE update_Logs (  
    id SERIAL PRIMARY KEY,  
    old_name TEXT,  
    new_name TEXT,  
    updated_at TIMESTAMP  
);  
  
CREATE TABLE delete_Logs (  
    id SERIAL PRIMARY KEY,  
    deleted_user TEXT,
```

);

*-- =====**-- BEFORE INSERT TRIGGER**-- =====**CREATE OR REPLACE FUNCTION before_insert_users()**RETURNS TRIGGER AS \$\$**BEGIN**NEW.created_at := NOW();**RETURN NEW;**END;**\$\$ LANGUAGE plpgsql;**CREATE TRIGGER trg_before_insert_users**BEFORE INSERT ON users**FOR EACH ROW**EXECUTE FUNCTION before_insert_users();**-- =====**-- AFTER INSERT TRIGGER**-- =====**CREATE OR REPLACE FUNCTION after_insert_users()**RETURNS TRIGGER AS \$\$**BEGIN**INSERT INTO user_logs(user_id, action, log_time)**VALUES (NEW.id, 'INSERT', NOW());*

END;

\$\$ LANGUAGE plpgsql;

CREATE TRIGGER trg_after_insert_users

AFTER INSERT ON users

FOR EACH ROW

EXECUTE FUNCTION after_insert_users();

-- =====

-- BEFORE UPDATE TRIGGER

-- =====

CREATE OR REPLACE FUNCTION before_update_users()

RETURNS TRIGGER AS \$\$

BEGIN

IF NEW.name IS NULL THEN

RAISE EXCEPTION 'Name cannot be NULL';

END IF;

NEW.updated_at := NOW();

RETURN NEW;

END;

\$\$ LANGUAGE plpgsql;

CREATE TRIGGER trg_before_update_users

BEFORE UPDATE ON users

FOR EACH ROW

```
EXECUTE FUNCTION before_update_users();

-- =====

-- AFTER UPDATE TRIGGER

-- =====

CREATE OR REPLACE FUNCTION after_update_users()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO update_logs(old_name, new_name, updated_at)
    VALUES (OLD.name, NEW.name, NOW());

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_after_update_users
AFTER UPDATE ON users
FOR EACH ROW
EXECUTE FUNCTION after_update_users();

-- =====

-- BEFORE DELETE TRIGGER

-- =====

CREATE OR REPLACE FUNCTION before_delete_users()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.name = 'admin' THEN
        RAISE EXCEPTION 'Cannot delete admin user';
```

```
END IF;

RETURN OLD;

END;

$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_before_delete_users
BEFORE DELETE ON users
FOR EACH ROW
EXECUTE FUNCTION before_delete_users();

-- =====
-- AFTER DELETE TRIGGER
-- =====

CREATE OR REPLACE FUNCTION after_delete_users()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO delete_logs(deleted_user, deleted_at)
    VALUES (OLD.name, NOW());

    RETURN OLD;
END;

$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_after_delete_users
AFTER DELETE ON users
FOR EACH ROW
EXECUTE FUNCTION after_delete_users();
```

```
-- =====  
  
-- STATEMENT LEVEL TRIGGER  
  
-- =====  
  
CREATE OR REPLACE FUNCTION statement_trigger_example()  
RETURNS TRIGGER AS $$  
  
BEGIN  
  
    RAISE NOTICE 'A statement executed on users table';  
  
    RETURN NULL;  
  
END;  
  
$$ LANGUAGE plpgsql;  
  
  
CREATE TRIGGER trg_statement_users  
AFTER INSERT ON users  
FOR EACH STATEMENT  
EXECUTE FUNCTION statement_trigger_example();  
  
  
-- =====  
  
-- INSERT SAMPLE DATA  
  
-- =====  
  
INSERT INTO users(name) VALUES ('Ankush');  
INSERT INTO users(name) VALUES ('Rahul');  
INSERT INTO users(name) VALUES ('admin');  
  
  
-- =====  
  
-- UPDATE SAMPLE DATA  
  
-- =====  
  
UPDATE users SET name = 'Ankush Kumar' WHERE name = 'Ankush';
```

```
-- =====  
  
-- DELETE SAMPLE DATA  
  
-- =====  
  
DELETE FROM users WHERE name = 'Rahul';  
  
  
-- This will FAIL (trigger demo)  
-- DELETE FROM users WHERE name = 'admin';  
  
  
-- =====  
  
-- VIEW RESULTS  
  
-- =====  
  
SELECT * FROM users;  
  
SELECT * FROM user_Logs;  
  
SELECT * FROM update_Logs;  
  
SELECT * FROM delete_Logs;
```

Output:

```
ERROR: Cannot delete admin user  
CONTEXT: PL/pgSQL function before_delete_users() line 4 at RAISE  
  
SQL state: P0001
```

Figure 1: Deleting admin (delete_admin_log.png)

	id [PK] integer	name text	created_at timestamp without time zone	updated_at timestamp without time zone
1	3	admin	2026-04-21 16:11:36.901996	[null]
2	1	Ankush Kum...	2026-04-21 16:08:33.733407	2026-04-21 16:12:18.763797

Figure 2: First select query (select_users.png)

	log_id [PK] integer	user_id integer	action text	log_time timestamp without time zone
1	1	1	INSERT	2026-04-21 16:08:33.733407
2	2	2	INSERT	2026-04-21 16:11:30.073345
3	3	3	INSERT	2026-04-21 16:11:36.901996

Figure 3: Second select query for user_logs (select_userlogs.png)

	id [PK] integer	old_name text	new_name text	updated_at timestamp without time zone
1	1	Ankush	Ankush Kum...	2026-04-21 16:12:18.763797

Figure 4: Third select query for update logs (select_updatelogs.png)

	id [PK] integer	deleted_user text	deleted_at timestamp without time zone
1	1	Rahul	2026-04-21 16:12:45.309067

Figure 5: Fourth select query for delete_logs (select_deletelogs.png)

Learning outcomes (What I have learnt):

1. I learned how to enforce data integrity at the database level using triggers, such as preventing invalid updates (like NULL values) and restricting deletion of important records (e.g., admin users).
2. I understood how to automate repetitive tasks like setting timestamps (created_at, updated_at) without relying on application code, making the system more reliable and consistent.
3. I learned how to track all important database operations (INSERT, UPDATE, DELETE) using triggers and maintain logs, which is useful for debugging, monitoring, and real-world applications.
4. I gained clarity on when to use BEFORE, AFTER, and STATEMENT-level triggers, and how each serves a different purpose in handling database events efficiently.